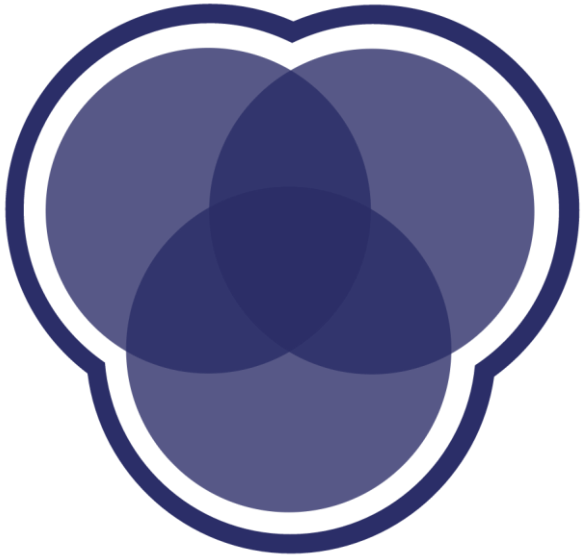


Distributed Downloader

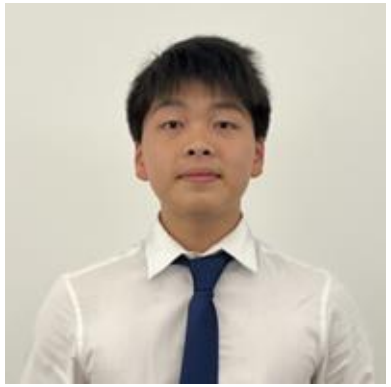
Showcase Presentation

Cornell Data Science
Spring 2026

Agenda

1. Introduction
2. Motivation + Background
3. Architecture
4. Demo
5. Results
6. Challenges + Next Steps

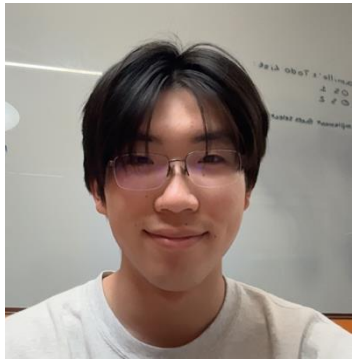
Meet the team!



Naijei Jiang
Tech Lead | CS ‘28

Harshaan Chugh
Project Manager | CS '28

Tanvi Bhave
CS '99



Eric Sun
Math+CS ‘29

Skai Nzeuton
CS ‘27

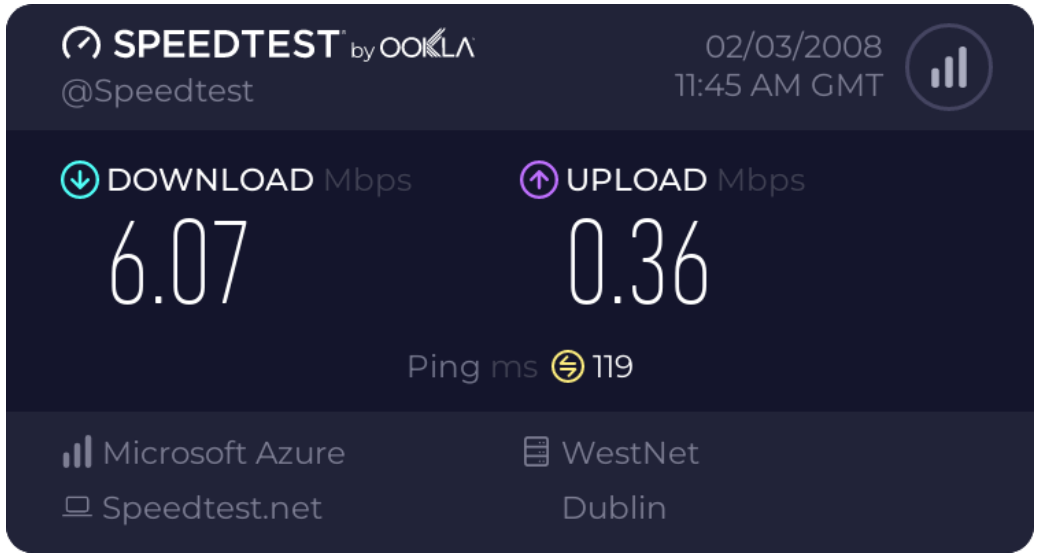
Yitbrek Mata
CS ‘29

Motivation + Background

Motivation

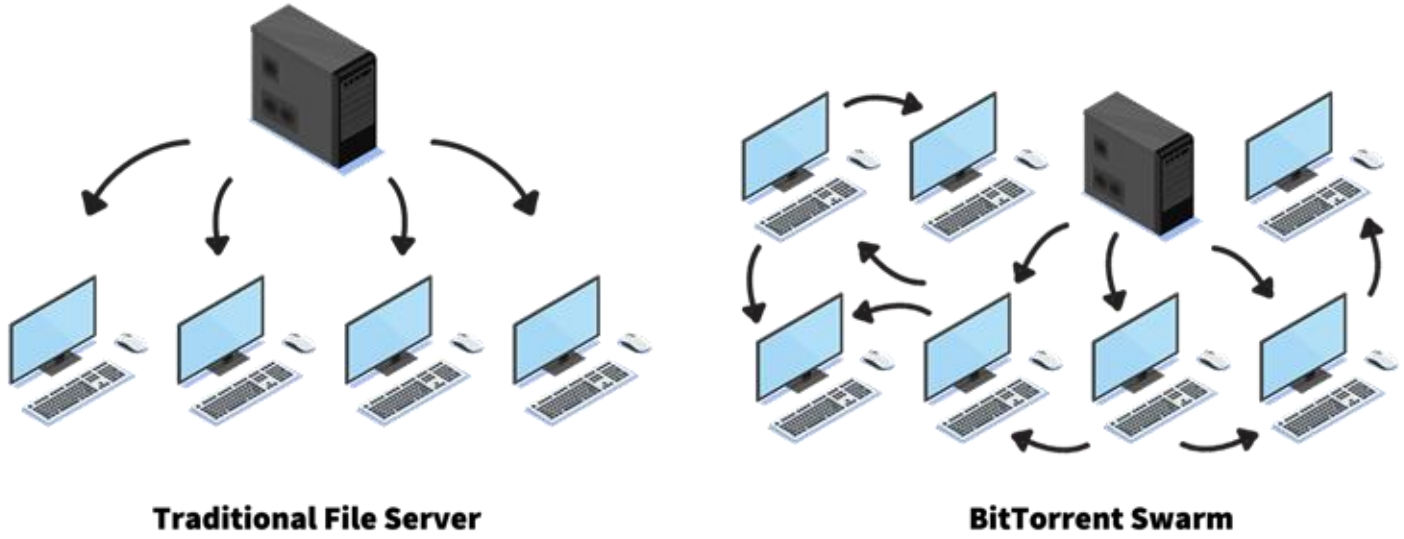
- Slow downloading times?
  - Even if the people around you have the files?

- Distributed Downloader!
  - Inspired by BitTorrent



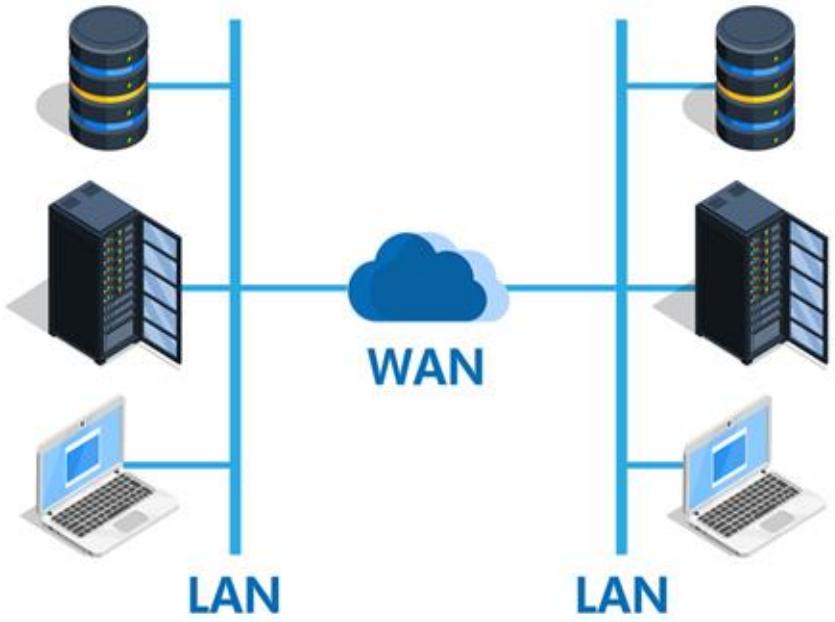
Intro to BitTorrent

- No central server, continuously growing upload capacity
- Protocol shares data through P2P connections
  - Tracker keeps track of peers, but chunks are shared P2P directly
- Efficient file distribution and fault tolerance



Local Area Network

Environment	Single Connection	8-Thread Parallel	32-Threads
LAN (Local)	~117 Mbps	~137 Mbps	~114 Mbps
WAN (Sweden - High Latency)	~7 Mbps	~42 Mbps	~71.44 Mbps
WAN (Newark, NJ - Low Latency)	~35 Mbps	~72 Mbps	~111.68 Mbps
Speedtest.net (Ultra-Low Latency)	N/A	93.19 Mbps	N/A



Architecture

gRPC

What is it?

- Google's communication framework
- Lets one service call functions on another service like a normal method call.

Current Setup

- Uses .proto files to define the services, methods, requests, and responses.
- Automatically generates Java code for the tracker, peer, and client to use

```
message PeerEndpoint {
  string id = 1;      // UUID string
  string ip = 2;      // e.g. "192.168.1.12"
  uint32 port = 3;    // peer gRPC port (e.g., 6001)
}
```

```
service Peer {
  rpc GetAvailability(FileRequest) returns (ChunkBitmap);
  rpc GetChunk(ChunkRequest) returns (ChunkResponse);
  rpc GetChunks(MultiChunkRequest) returns (stream ChunkResponse);
}
```

SpringBoot

What is it?

- A Java framework for building applications and services quickly
- Helps create REST APIs, web servers, and backend services with less setup

Current Setup

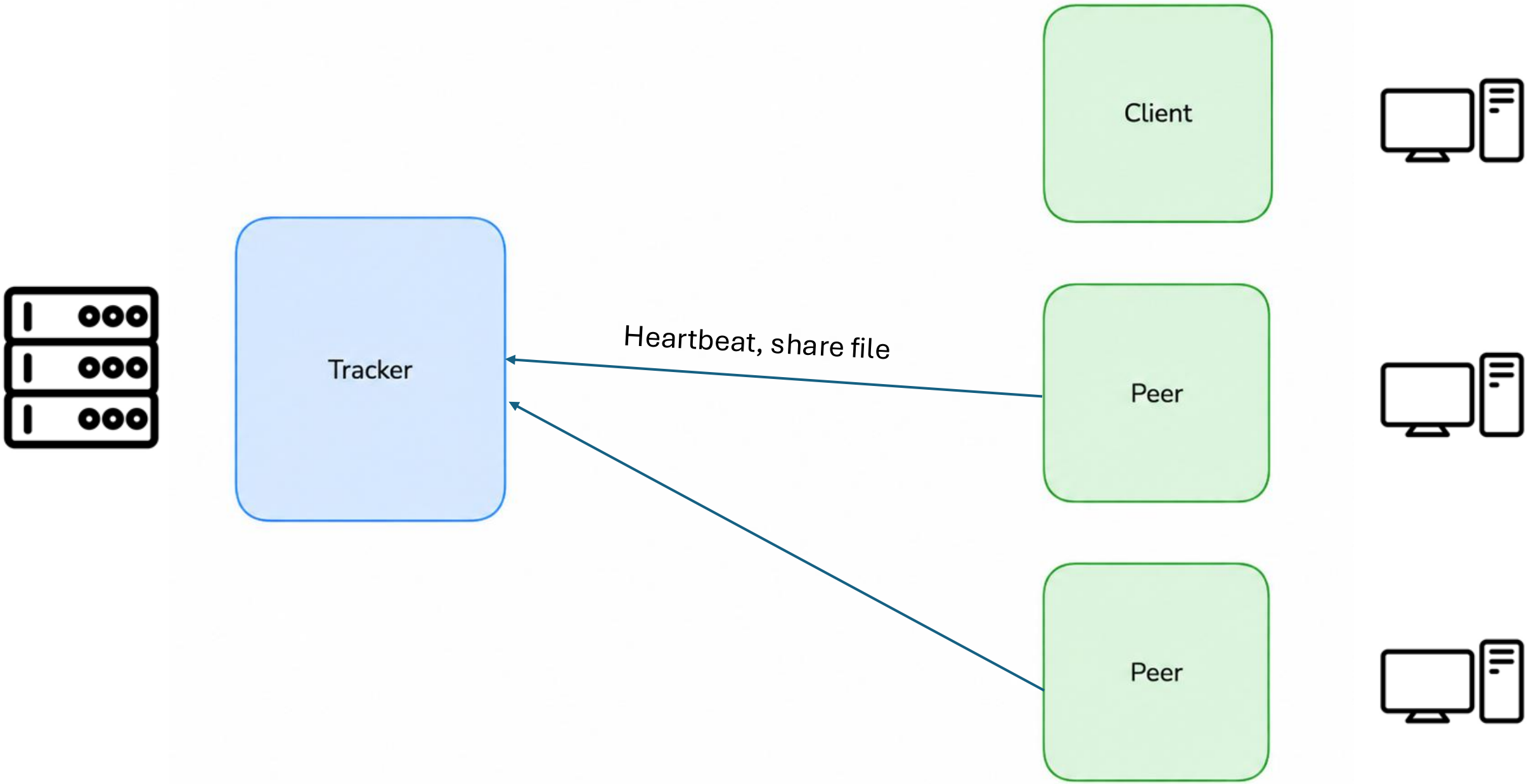
- Uses @SpringBootApplication to start the tracker and peer Spring Boot applications
- Uses @Service to register TrackerGrpcService and PeerGrpcService, so Spring Boot can run the logic for peer heartbeats, file tracking, and chunk sharing

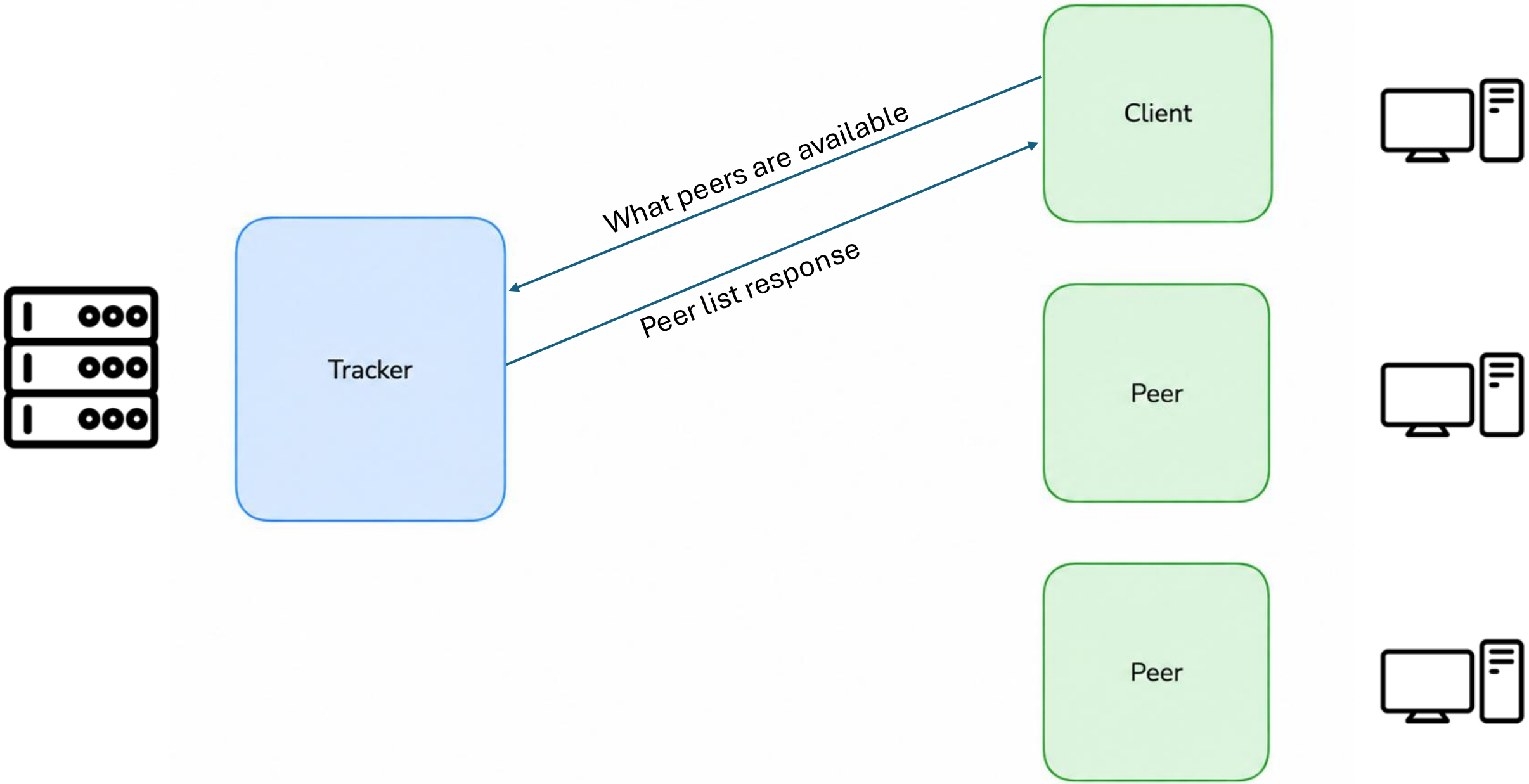
```
@Scheduled(fixedRate = 5000)
public void sendHeartbeat() {
    try {
        seedSharedFile();
```

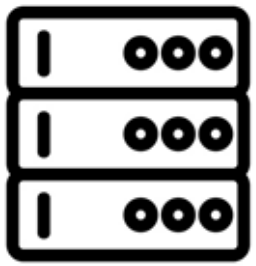
```
@Service
public class PeerGrpcService extends PeerGrpc.PeerImplBase {

    private static final int DEFAULT_CHUNK_SIZE = 4 * 1024 * 1024;
    private static final String HASH_ALGORITHM = "SHA-256";

    // run with --peer.share-file=/Users/you/Desktop/Test1mb.bin
    @Value("${peer.share-file:}")
    private String shareFilePath;
```


Client
Tracker
Heartbeat, share file
Peer
Peer


Client
What peers are available
Peer list response
Tracker
Peer
Peer



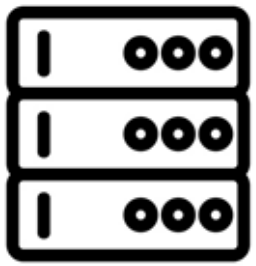
Tracker

Can I have some file chunks

Client

Peer

Peer



Tracker

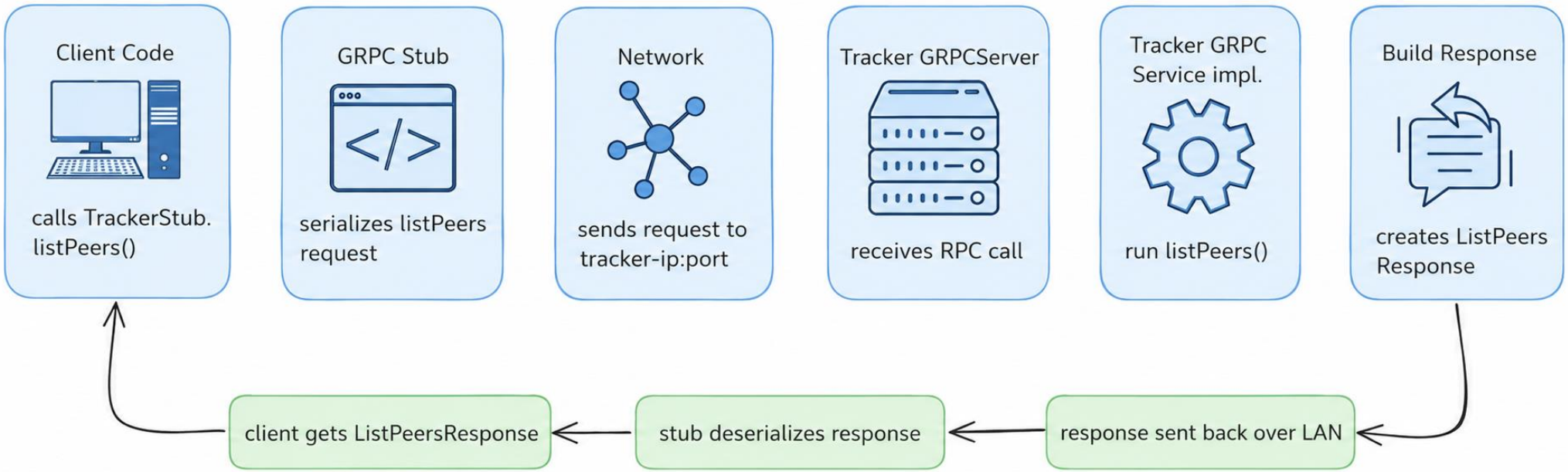
Here is the chunk

Client

Peer

Peer

gRPC in Action



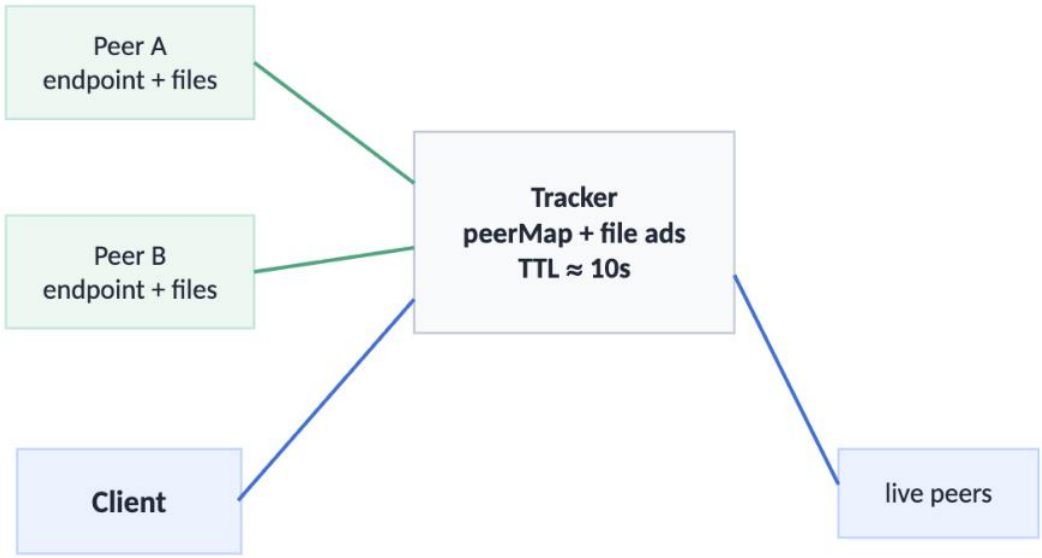
Tracker Messages

1. HeartbeatRequest
   - Peer registers itself and advertises file manifests.
   - Tracker keeps track of who's alive.

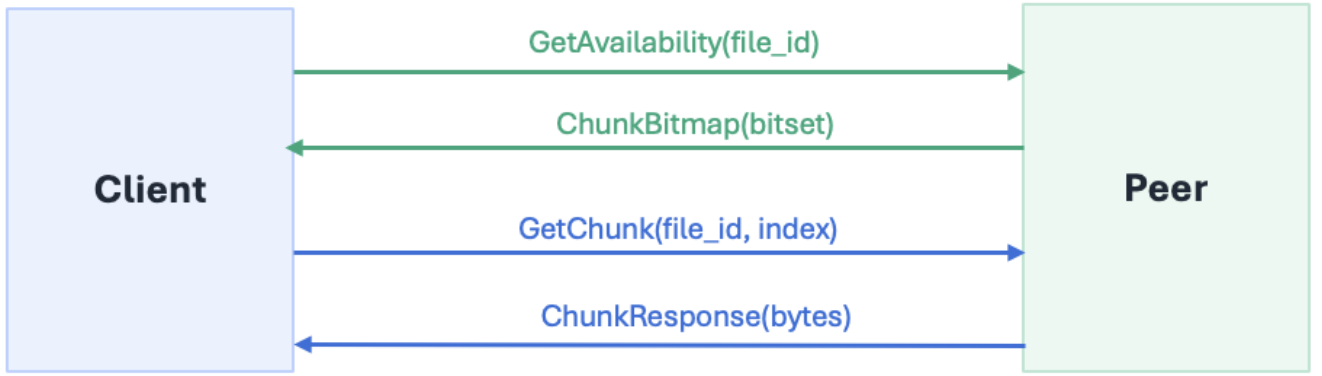
2. GetFileManifestRequest
   - Client needs file metadata → tracker returns information about chunks.

3. ListPeersRequest
   - Client asks for currently alive peers → tracker deletes stale peers & returns live ones.

Encapsulated messages:
- FileManifestEntry: file metadata
- PeerEndpoint: peer ID, peer's IP, and peer's port.
- Ack



Peer Messages



1. FileRequest / ChunkBitmap
   - Client asks a peer what chunks it has for a file.
   - Peer returns a bitmap showing which chunk indices are available.

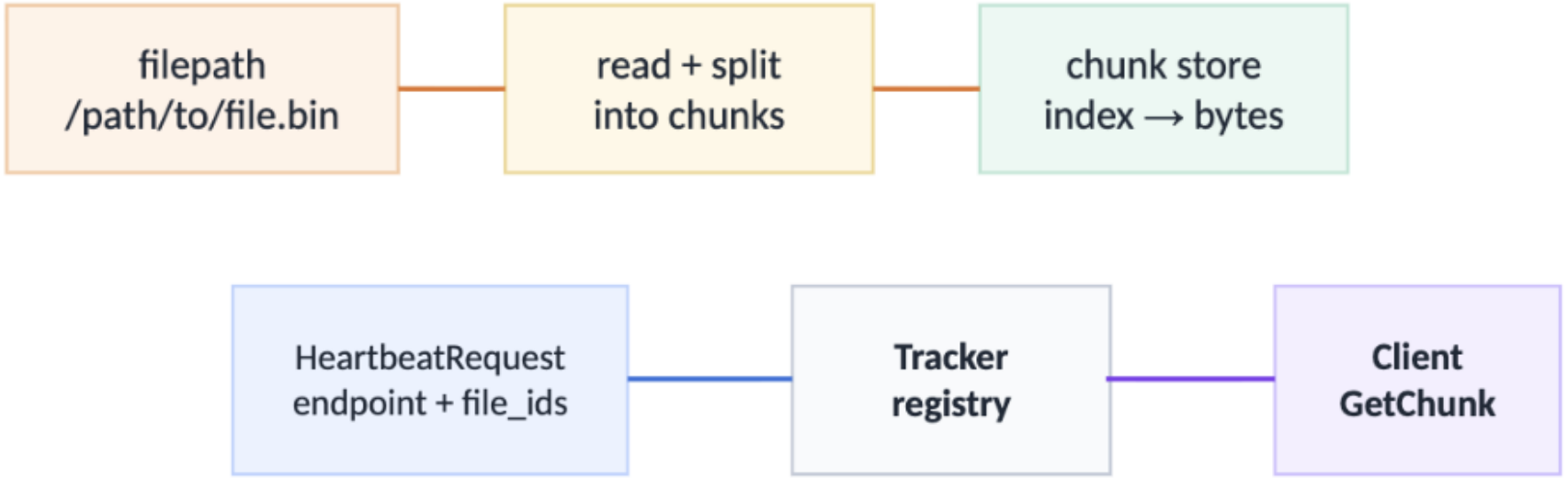
bitset example: peer has chunks 0, 2, 3, 7

2. ChunkRequest / ChunkResponse
   - Client asks a peer for one specific chunk.
   - Peer returns raw bytes for the requested chunk

3. MultiChunkRequest + streaming ChunkResponse
   - Client asks a peer for multiple chunks in one request.
   - Peer streams back ChunkResponses, one per chunk.

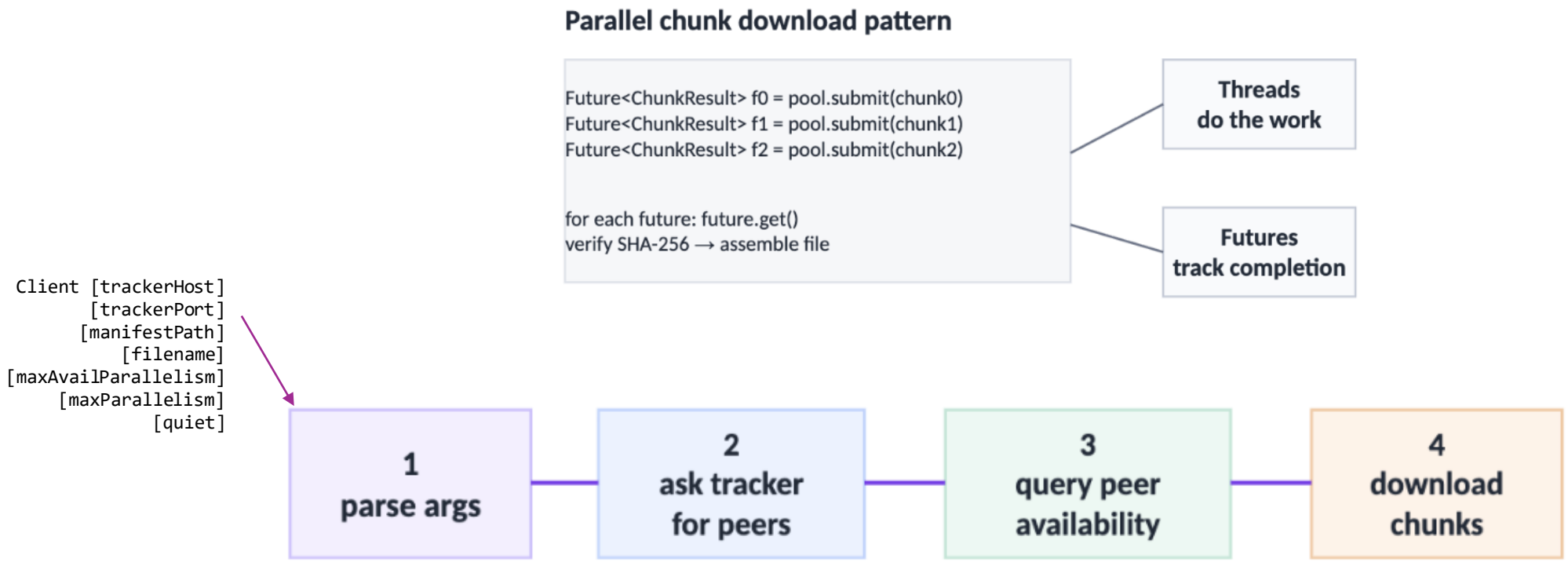
Peer Sharing Files from filepath

- The Makefile passes a flag SHARE_FILE into the peer as –peer.share-file=...
- Peer advertises its address and file it's sharing to the tracker in heartbeat
- Chunks in shared files are stored in a map on the peer's side before client requests a chunk.



Client is the Orchestrator (No messages)

- Futures are used to track asynchronous chunk-download tasks submitted to the thread pool.
- One future represents one download job.
- Client later waits on these futures (barrier-style) before assembling the final file, which ensures that all chunks are downloaded and lets us catch errors.



Demo

Results

Results

Averaging ~20 Mbps download speed

Concurrently download and reconstruct file chunks

Support arbitrary number of peers

Tracker hosted on compute cluster

Challenges + Next Steps

Challenges

OVERHEAD

- gRPC
  - Channel creation
  - Message metadata
- Network throttling

Next Steps

Move to communication framework with less overhead

- Direct data transfer using TCP
- C++??

Combine peer and client processes

Download arbitrary files requested by client

Thanks for listening!
Questions?